

MIGUEL ANGEL SALAZAR MARTINEZ

Universidad Politécnica de San Luis Potosi

CNO 4 – Seguridad Informática

Profesor: Servando López Contreras

Actividad 10 – File Path Traversal | Traversal
sequences with superfluous url-decode

23/09/26

Introducción

En el ámbito de la ciberseguridad, el juego entre ataque y defensa a menudo se decide en los detalles. En las secciones anteriores hemos visto cómo evadir filtros simples mediante rutas absolutas y cómo aprovechar eliminaciones no recursivas para reconstruir secuencias prohibidas. Ahora, el escenario que nos ocupa presenta una defensa aparentemente más robusta: la aplicación bloquea las secuencias de path traversal e incluso realiza una decodificación URL adicional sobre la entrada. Sin embargo, cuando el orden de estas operaciones es incorrecto, el atacante puede aprovechar la doble codificación para que la secuencia maliciosa "aparezca" ante el sistema solo después de haber pasado los controles de seguridad. Este caso demuestra que el orden en que se validan y decodifican los datos es tan importante como la validación misma, y que la defensa debe realizarse siempre después de la canonicalización completa de la entrada.

Marco Teórico

El mecanismo de la decodificación superflua

El núcleo de este laboratorio es el "URL-decode superfluo". La aplicación, al recibir el nombre de archivo, bloquea cualquier entrada que contenga secuencias de path traversal (../). Sin embargo, después de aplicar este filtro, realiza una decodificación URL adicional sobre la entrada antes de usarla. Esto significa que si el atacante envía una secuencia que, al ser decodificada una primera vez, no parece contener ../, el filtro la dejará pasar. Pero la segunda decodificación (realizada por la aplicación) la transformará en la secuencia peligrosa.

El principio de la doble codificación

El atacante puede construir una carga útil como `..%252f..%252f..%252fetc/passwd`. Aquí, %25 es el código URL del carácter %. Cuando la aplicación realiza su decodificación, %25 se convierte en %, y la cadena queda como `..%2f..%2f..%2fetc/passwd`. En ese momento, el filtro no detecta ../ porque la secuencia todavía está codificada como %2f. Sin embargo, la aplicación luego realiza otra decodificación (o el sistema operativo la realiza al abrir el archivo), convirtiendo %2f en /, resultando en `../../etc/passwd`.

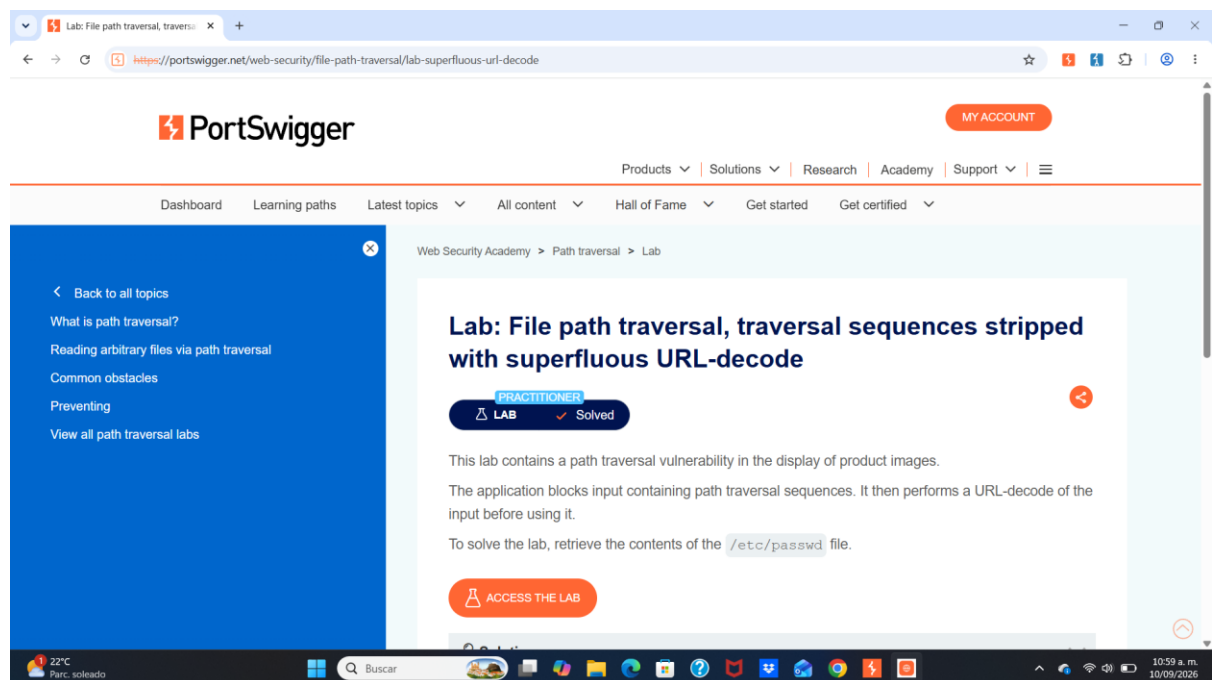
La contradicción fundamental: validar antes de decodificar

Este bypass expone un error de orden crítico: la validación debe realizarse después de que la entrada haya sido completamente decodificada y canonicalizada. Si la aplicación valida la cadena tal como llega (codificada), y luego la decodifica antes de usarla, está validando algo diferente a lo que finalmente se usará. CVE-2021-41773 en Apache fue un ejemplo real de este error: el servidor validaba la ruta antes de decodificarla completamente, permitiendo que %2e%2e pasara el filtro y luego se resolviera como `..` al abrir el archivo. La lección es clara: decodificar completamente, normalizar, y solo entonces validar.

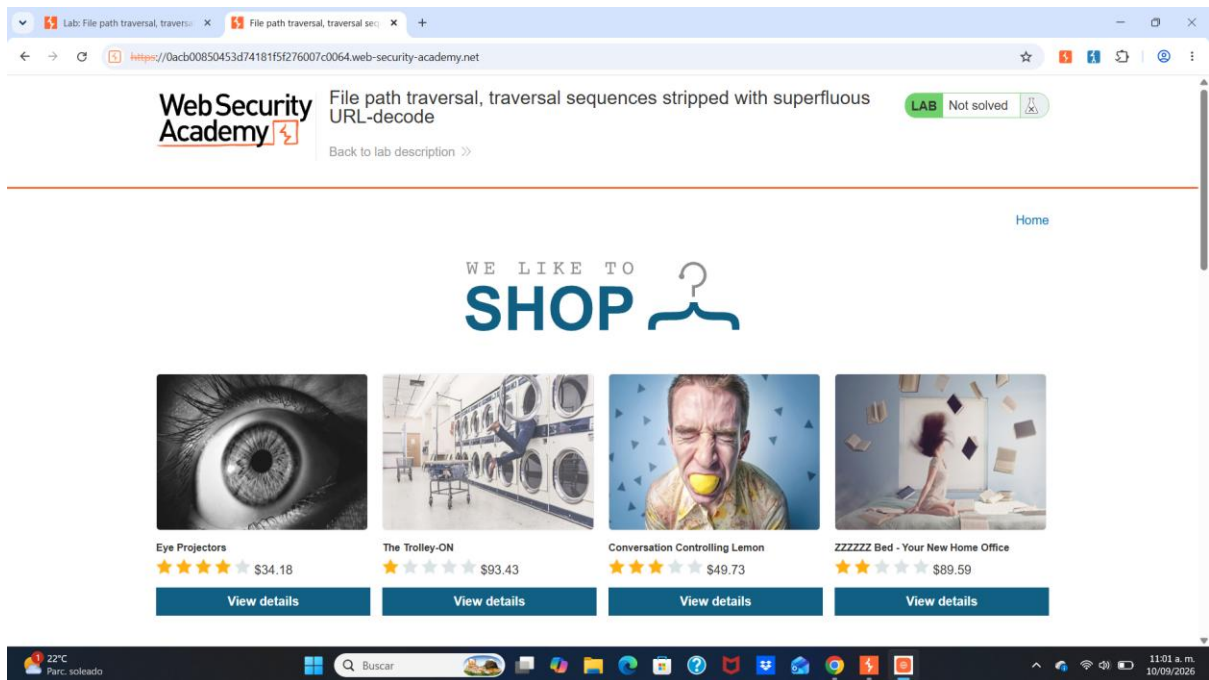
Posibles Soluciones

Para evitar esta vulnerabilidad, la defensa no debe validar la entrada antes de decodificarla, ya que un atacante puede usar doble codificación (%252f) para que la secuencia maliciosa aparezca solo después de pasar el filtro. La solución correcta consiste en decodificar completamente la entrada primero, luego normalizar la ruta con `realpath()` y, finalmente, verificar que la ruta resultante permanezca dentro del directorio permitido. Además, se recomienda implementar una lista blanca de archivos permitidos, usar identificadores numéricos en lugar de nombres de archivo, y aplicar permisos restrictivos para limitar el impacto en caso de fallo. La clave es validar siempre la representación final y canonicalizada de la entrada, nunca una versión intermedia que aún puede transformarse.

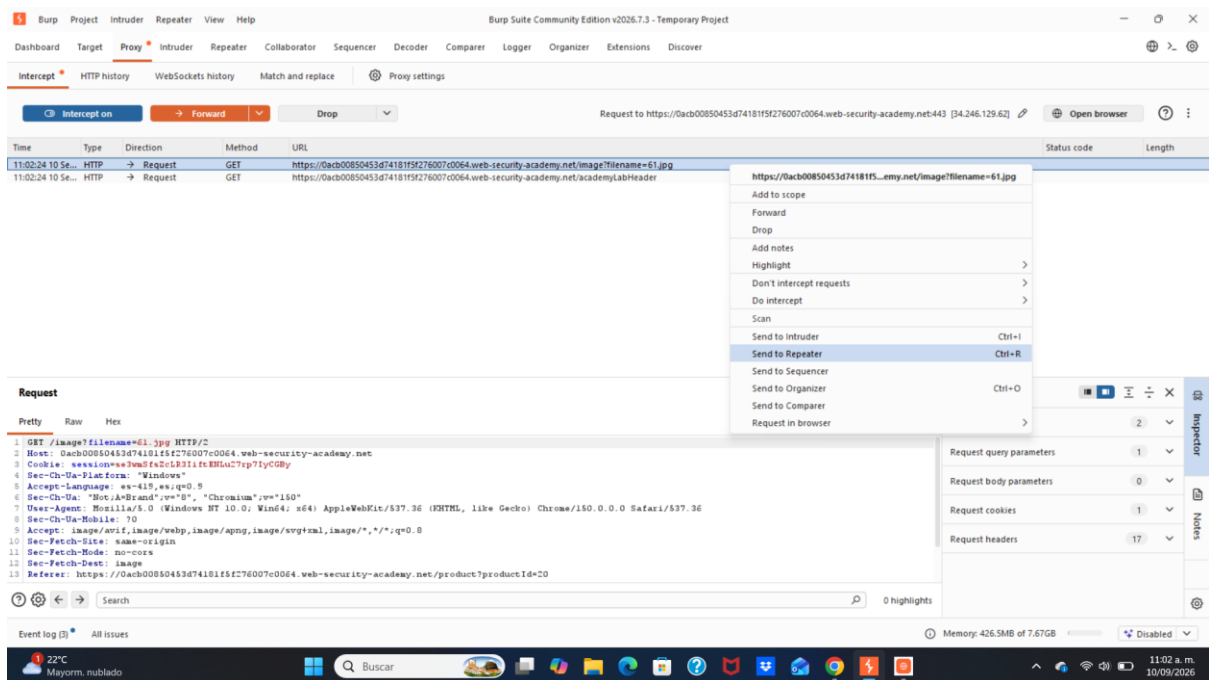
Desarrollo



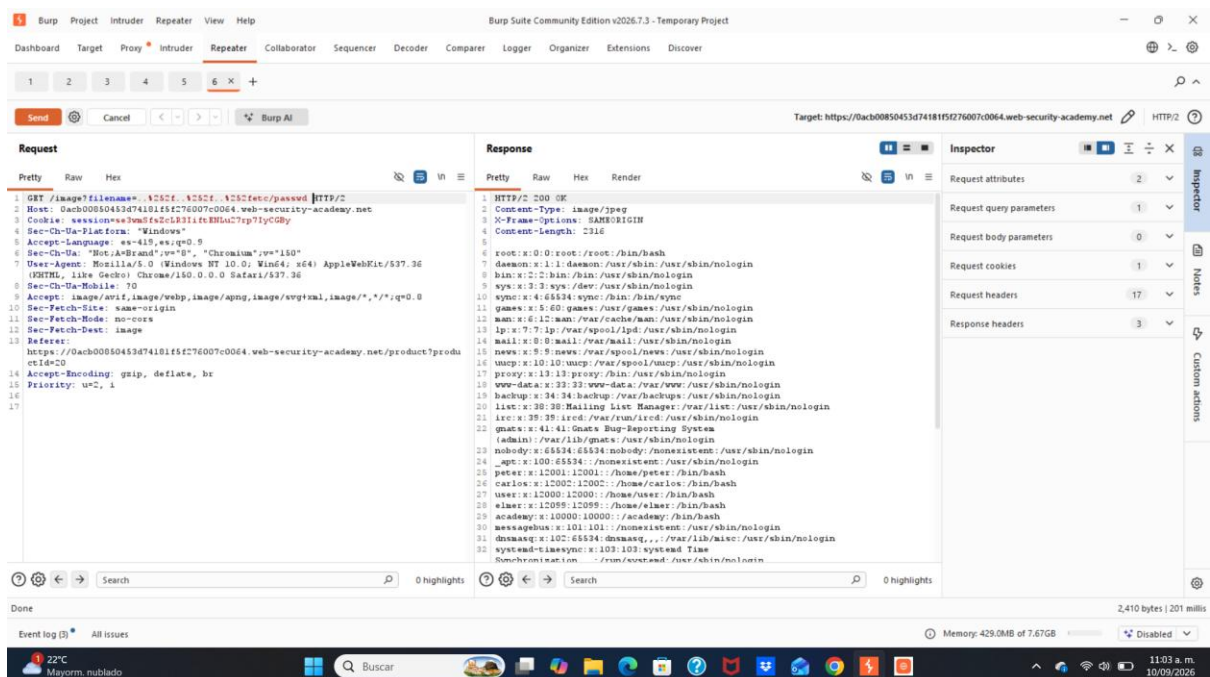
Copiamos la URL de la actividad en el navegador que nos proporciona el burpsuite para abrir la actividad de traversal stripped with superfluos URL-decode



Una vez nos aparezca la página por la que estaremos aprovechándonos de la vulnerabilidad path traversal, daremos click en cualquiera de las imágenes en la parte de “View details”.



Cuando nos empiece a llegar tráfico de la página, nos iremos a la parte de proxy en nuestro burpsuit y a los logs que nos aparezcan les daremos click en forward hasta llegar al log que nos interesa que es el de tipo imagen “jpg”, al cual enviaremos al repetidor para proceder a analizarlo.



Continuaremos en la sección del repetidor que es donde analizaremos este log, nos dirigiremos a la parte Request en filename, y borraremos lo que se encuentre ahí, y procederemos a ingresar `../../../../etc/passwd` que es la vulnerabilidad por la que estaremos obteniendo a los usuarios y passwords, a esta vulnerabilidad se le conoce como “Doble decodificación” el SO decodifica el `%25` que nos da como resultado un `%` quedando `../../../../etc/passwd` que después se vuelve a decodificar que nos da un `./.`, que juntando ya todo decodificado nos queda `../../../../etc/passwd` que es la ruta a la que finalmente tenemos acceso.

Conclusión

Este laboratorio ofrece una lección fundamental sobre el diseño de defensas de seguridad: el orden de las operaciones de validación y decodificación es crítico. No basta con tener un filtro que bloquee ../; ese filtro debe operar sobre la representación final y canonicalizada de la entrada, no sobre una versión intermedia que puede transformarse en algo peligroso después de pasar el control.

La doble codificación es una técnica clásica de evasión de filtros que aprovecha precisamente esta discrepancia entre lo que se valida y lo que se usa. La solución correcta consiste en decodificar completamente, canonicalizar la ruta (resolviendo ../ y .), y solo entonces aplicar las validaciones (como verificar que la ruta esté dentro del directorio permitido). Cualquier defensa que valide antes de la canonicalización completa es vulnerable a este tipo de bypass. Este principio se aplica no solo al path traversal, sino a cualquier validación de entrada en general: validar el resultado final, no la entrada cruda.